



UNIVERSIDAD AUTÓNOMA DE SAN LUIS POTOSÍ

FACULTAD DE ESTUDIOS
PROFESIONALES
ZONA MEDIA

INGENIERÍA MECATRÓNICA



PRÁCTICA 9

**Contador Bidireccional 0–99 con
Multiplexeo de Dos Displays de 7 Segmentos**

MATERIA: MICROCONTROLADORES

DOCENTE: Ing. JESÚS PADRÓN

ALUMNO: CARLOS ANGEL GARCÍA SIFUENTES

CLAVE: A383917

CARRERA: INGENIERÍA MECATRÓNICA

SEMESTRE: 4° SEMESTRE

FECHA: 24 DE MAYO DE 2026

Contador Decimal Bidireccional de 0 a 99 con Multiplexeo

de Dos Displays de 7 Segmentos en Raspberry Pi Pico W

CARLOS ÁNGEL GARCÍA SIFUENTES

a383917@alumnos.uaslp.mx

Fecha: 20 de mayo de 2026

Resumen—Se presenta el desarrollo de la Práctica 9, que extiende el trabajo previo con displays de 7 segmentos al incorporar un segundo display y la técnica de *multiplexeo por tiempo* para visualizar simultáneamente las decenas y las unidades de un contador decimal de 0 a 99. El programa, escrito en lenguaje C con el SDK de Raspberry Pi, alterna la activación de cada display a 5 ms por dígito, logrando una frecuencia de refresco de 100 Hz que el ojo humano percibe como encendido continuo. Un pulsador con resistencia *pull-up* interna e *antirrebote* por software invierte el sentido del conteo (ascendente/descendente) en el instante de la pulsación. El valor actual se imprime además en el monitor serial de VS Code mediante `printf`.

I. INTRODUCCIÓN

I-A Descripción General de la Práctica

Esta práctica amplía la Práctica 8 en dos aspectos fundamentales. El primero es la incorporación de un segundo display de 7 segmentos para mostrar valores de dos dígitos (decenas y unidades), lo que eleva a nueve el número de pines GPIO requeridos solo para el display (siete de segmentos + dos de control de dígito). El segundo es la técnica de **multiplexeo temporal**, que permite controlar ambos displays con los mismos siete pines de segmento compartidos, reduciendo drásticamente el número de pines necesarios.

Adicionalmente, el botón ya no solo reinicia: al pulsarlo invierte el sentido del contador entre ascendente (0→99) y descendente (99→0), permaneciendo en el nuevo sentido hasta una nueva pulsación.

I-B Trabajo Previo: Multiplexeo de Displays

El **multiplexeo temporal** (*time-division multiplexing*) es una técnica en la que varios displays comparten los mismos pines de datos (segmentos), pero se activan de forma alternada y muy rápida, de manera que solo uno está encendido en cada instante. Si la alternancia ocurre a una frecuencia superior a aproximadamente 50 Hz, la

persistencia de la visión humana integra las imágenes y percibe ambos dígitos encendidos de forma continua y estable.

En esta práctica el ciclo de multiplexeo es de 10 ms por iteración completa (5 ms para las decenas + 5 ms para las unidades), repitiendo 40 iteraciones por cada actualización del contador, lo que da una pausa total de aproximadamente 400 ms entre incrementos o decrementos.

El control de qué display está activo se realiza mediante dos pines de habilitación (GPIO 10 y GPIO 11) que gobiernan la línea de cátodo común de cada display: al poner en alto el pin correspondiente se activa ese dígito mientras el otro permanece apagado.

I-C Monitor Serial en VS Code

El código incluye llamadas a `printf()` para imprimir el valor actual del contador y el sentido del conteo en el monitor serial. Para visualizarlos en Pico – Visual Studio Code es necesario abrir la terminal integrada y conectarse al puerto COM asignado a la Pico W, o bien usar la extensión **Serial Monitor** que permite ver la salida de `stdio` directamente dentro del editor. Esto es posible porque `stdio_init_all()` redirige la salida estándar al puerto USB virtual que crea la placa al conectarse.

II. DESARROLLO

II-A Material y Recursos Utilizados

- **Hardware:** 1 Placa Raspberry Pi Pico W, 1 cable micro USB tipo B, 2 displays de 7 segmentos de cátodo común, resistencias de 220 Ω (una por segmento o una compartida), 1 pulsador sin enclavamiento mecánico y 1 protoboard.
- **Software y librerías:** SDK de Raspberry Pi Pico, Pico – Visual Studio Code, extensión Serial Monitor, librerías `pico_stdlib` y `hardware/gpio.h`.

II-B Asignación de Pines

La Tabla I resume la asignación de todos los pines GPIO utilizados en la práctica.

Tabla I: Asignación de pines GPIO para la Práctica 9

GPIO	Función	Notas
2	Segmento a	Compartidos por ambos displays
3	Segmento b	
4	Segmento c	
5	Segmento d	
6	Segmento e	
7	Segmento f	
8	Segmento g	
9	Botón pulsador	Pull-up interno, activo en bajo
10	Habilitación Display 1 (decenas)	Control de cátodo común
11	Habilitación Display 2 (unidades)	Control de cátodo común

II-C Estructura del Proyecto y Configuración de CMake

ARCHIVO CMakeLists.txt

```
cmake_minimum_required(VERSION 3.13)

include(pico_sdk_import.cmake)
set(PICO_BOARD pico_w)

project(PRACTICA_9)

pico_sdk_init()

add_executable(Displays
    Displays.c
)

target_link_libraries(Displays
    pico_stdlib
)

pico_add_extra_outputs(Displays)
```

II-D Implementación del Código en C

El archivo `Displays.c` se organiza en cuatro bloques bien diferenciados: la tabla de patrones de segmentos, dos funciones auxiliares *inline* para escribir segmentos

y habilitar dígitos, la función de detección de *click* con antirrebote, y el lazo principal con el multiplexeo.

CÓDIGO FUENTE – Displays.c

```
#include <stdio.h>
#include "pico/stdlib.h"
#include "hardware/gpio.h"

#define FIRST_GPIO 2
#define BUTTON_GPIO 9
#define DIG1_GPIO 10
#define DIG2_GPIO 11
#define DEBOUNCE_MS 40

// Patrones de segmentos para digitos 0-9
int bits[10] = {
    0x3f, 0x06, 0x5b, 0x4f, 0x66,
    0x6d, 0x7d, 0x07, 0x7f, 0x67
};

// Escribe el patron de segmentos de un
// dígito en GPIO 2..8
static inline void segments_write(int digit) {
    uint32_t seg_all = (0x7ful <<
        FIRST_GPIO);
    uint32_t seg_num = ((uint32_t)bits[
        digit]) << FIRST_GPIO;
    gpio_clr_mask(seg_all);
    gpio_set_mask(seg_num);
}

// Activa/desactiva cada display (c todo
// com n)
static inline void dig_enable(bool d1, bool
    d2) {
    gpio_put(DIG1_GPIO, d1);
    gpio_put(DIG2_GPIO, d2);
}

// Antirrebote por tiempo: detecta un
// nico "click" por pulsaci n
static bool button_clicked(void) {
    static bool last_stable = true;
    static bool last_read = true;
    static absolute_time_t last_change;

    bool now = gpio_get(BUTTON_GPIO);
    if (now != last_read) {
        last_read = now;
        last_change = get_absolute_time();
    }
    if (absolute_time_diff_us(last_change,
        get_absolute_time()) > (
        DEBOUNCE_MS * 1000)) {
        if (now != last_stable) {
            bool prev = last_stable;
            last_stable = now;
            if (prev == true && last_stable
```

```

        == false)
        return true;    // flanco
                        de bajada confirmado
    }
}
return false;
}

int main() {
    stdio_init_all();

    // Segmentos GP2..GP8 como salidas
    for (int gp = FIRST_GPIO; gp <
        FIRST_GPIO + 7; gp++) {
        gpio_init(gp);
        gpio_set_dir(gp, GPIO_OUT);
        gpio_put(gp, 0);
    }

    // Pines de habilitacion de displays
    gpio_init(DIG1_GPIO); gpio_set_dir(
        DIG1_GPIO, GPIO_OUT);
    gpio_init(DIG2_GPIO); gpio_set_dir(
        DIG2_GPIO, GPIO_OUT);
    dig_enable(false, false);

    // Boton con pull-up interno
    gpio_init(BUTTON_GPIO);
    gpio_set_dir(BUTTON_GPIO, GPIO_IN);
    gpio_pull_up(BUTTON_GPIO);

    int value = 0;
    bool up    = true;    // true = sube,
                        false = baja

    while (true) {
        if (button_clicked()) up = !up;
        // inversion inmediata

        int tens = value / 10;
        int ones = value % 10;

        // 40 ciclos de refresco (~400 ms
        // totales, 100 Hz por dígito)
        for (int i = 0; i < 40; i++) {
            if (button_clicked()) up = !up;

            // Decenas
            segments_write(tens);
            dig_enable(true, false);
            sleep_ms(5);
            dig_enable(false, false);

            // Unidades
            segments_write(ones);
            dig_enable(false, true);
            sleep_ms(5);
            dig_enable(false, false);
        }

        // Actualizar contador

```

```

        if (up) value = (value == 99) ? 0
            : value + 1;
        else value = (value == 0) ? 99
            : value - 1;

        printf("Numero: %d | Sentido: %s\n"
            , value, up ? "SUBE" : "BAJA");
    }
}

```

II-D1. Función `segments_write()`

Esta función auxiliar aplica el patrón de segmentos de un dígito sobre los GPIO 2–8 en dos pasos: primero limpia todos los segmentos con `gpio_clr_mask()` aplicando una máscara de 7 bits desplazada a la posición `FIRST_GPIO`, y después activa únicamente los segmentos necesarios con `gpio_set_mask()`. Esto garantiza que no haya segmentos residuales de la iteración anterior.

II-D2. Función `dig_enable()`

Controla cuál de los dos displays está activo en cada instante. Al poner en alto el pin de habilitación del display correspondiente, se cierra el circuito de su cátodo común y el display muestra el patrón presente en los pines de segmento. El otro display permanece con su cátodo abierto y no ilumina ningún segmento, aunque comparte los mismos pines de datos.

II-D3. Función `button_clicked()` – Antirrebote por tiempo

A diferencia del antirrebote de espera activa usado en prácticas anteriores, esta función implementa un antirrebote **no bloqueante** basado en marcas de tiempo (`absolute_time_t`). Guarda en variables estáticas el último estado leído y el último estado estable, registrando el momento en que detecta un cambio. Solo acepta el nuevo estado como válido si han transcurrido al menos 40 ms desde que inició la transición. De esta forma el lazo principal nunca se detiene a esperar: puede continuar refrescando el display mientras vigila el botón con total responsividad.

II-E Diagrama de Conexiones y Proceso de Compilación

El circuito se arma sobre una protoboard conectando los siete segmentos de ambos displays a los GPIO 2–8 (con una resistencia de 220 Ω en el pin común de cada display), los cátodos comunes a los GPIO 10 y 11 respectivamente, y el pulsador entre GPIO 9 y GND.

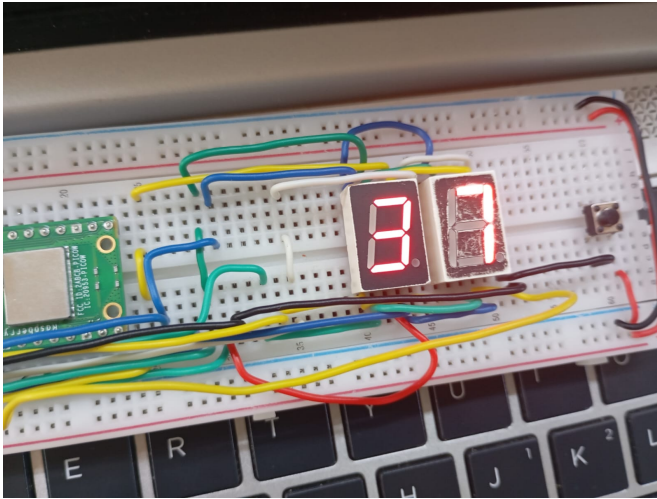


Figura 1: Diagrama en protoboard: dos displays de 7 segmentos multiplexados en GPIO 2–8, habilitación en GPIO 10–11 y pulsador en GPIO 9.

Con el código escrito se abre **Pico Visual Studio Code**, se selecciona *Displays (Executable)* en el panel CMake y se compila. El archivo `Displays.uf2` generado en la carpeta `build` se transfiere al microcontrolador en modo BOOTSEL.

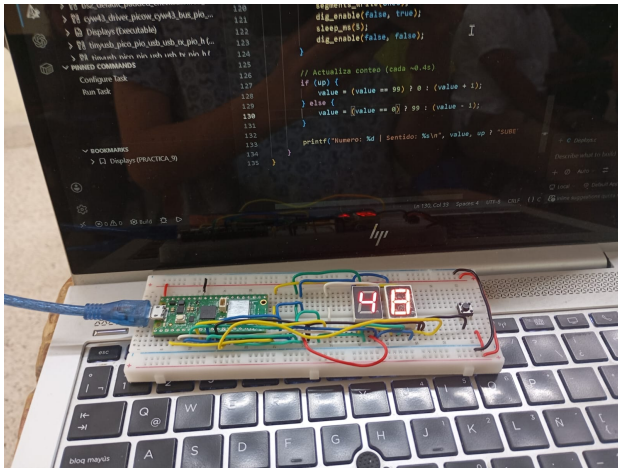


Figura 2: Proto con los displays de 7 segmentos frente al código.

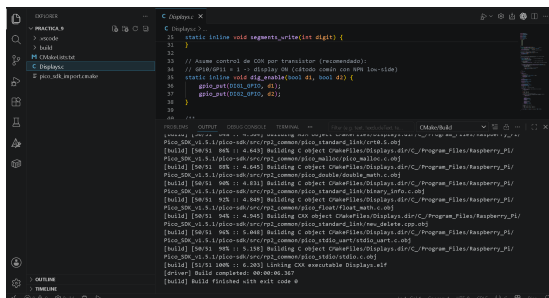


Figura 3: Monitor serial en VS Code mostrando el valor del contador y el sentido de conteo en tiempo real.

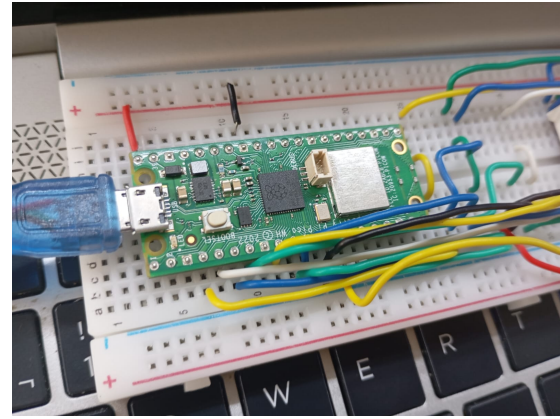


Figura 4: Circuito físico en funcionamiento con ambos displays mostrando un valor de dos dígitos.

III. CONCLUSIONES

- La técnica de multiplexeo temporal demostró ser una solución eficiente para controlar varios displays con un número reducido de pines GPIO: los siete pines de segmento son compartidos por ambos dígitos, y solo dos pines adicionales de habilitación determinan cuál display está activo en cada instante, reduciendo de 14 a 9 el total de pines requeridos.
- La frecuencia de refresco de 100 Hz por dígito (ciclos de 5 ms cada uno) resultó suficiente para que el ojo humano perciba ambos displays encendidos de forma continua y sin parpadeo visible, confirmando el principio de persistencia de la visión en aplicaciones embebidas.
- El antirrebote no bloqueante implementado en `button_clicked()` mediante marcas de tiempo supone una mejora significativa respecto al antirrebote por espera activa (`sleep_ms`) de prácticas anteriores, ya que el lazo de refresco del display nunca se interrumpe, manteniendo la visualización fluida incluso durante la detección de pulsaciones.
- La separación del código en funciones auxiliares (`segments_write()`, `dig_enable()`, `button_clicked()`) siguió el principio de responsabilidad única, haciendo el programa más legible, depurable y fácilmente extensible a tres o más displays con cambios mínimos.
- El monitor serial de VS Code se mostró como una herramienta de depuración valiosa: visualizar el valor del contador y el sentido de conteo en tiempo real permitió verificar el comportamiento del programa sin necesidad de instrumentación adicional, estableciendo una práctica de desarrollo que será útil en proyectos más complejos.

ANEXOS

A. Código fuente – Displays.c

ANEXO A – Displays.c (completo)

```
#include <stdio.h>
#include "pico/stdlib.h"
#include "hardware/gpio.h"

#define FIRST_GPIO 2
#define BUTTON_GPIO 9
#define DIG1_GPIO 10
#define DIG2_GPIO 11
#define DEBOUNCE_MS 40

int bits[10] = {
    0x3f, 0x06, 0x5b, 0x4f, 0x66,
    0x6d, 0x7d, 0x07, 0x7f, 0x67
};

static inline void segments_write(int digit) {
    uint32_t seg_all = (0x7Ful <<
        FIRST_GPIO);
    uint32_t seg_num = ((uint32_t)bits[
        digit]) << FIRST_GPIO;
    gpio_clr_mask(seg_all);
    gpio_set_mask(seg_num);
}

static inline void dig_enable(bool d1, bool
    d2) {
    gpio_put(DIG1_GPIO, d1);
    gpio_put(DIG2_GPIO, d2);
}

static bool button_clicked(void) {
    static bool last_stable = true;
    static bool last_read = true;
    static absolute_time_t last_change;
    bool now = gpio_get(BUTTON_GPIO);
    if (now != last_read) {
        last_read = now;
        last_change = get_absolute_time();
    }
    if (absolute_time_diff_us(last_change,
        get_absolute_time()) > (
        DEBOUNCE_MS * 1000)) {
        if (now != last_stable) {
            bool prev = last_stable;
            last_stable = now;
            if (prev == true && last_stable
                == false) return true;
        }
    }
    return false;
}

int main() {
    stdio_init_all();
```

```
for (int gp = FIRST_GPIO; gp <
    FIRST_GPIO + 7; gp++) {
    gpio_init(gp); gpio_set_dir(gp,
        GPIO_OUT); gpio_put(gp, 0);
}
gpio_init(DIG1_GPIO); gpio_set_dir(
    DIG1_GPIO, GPIO_OUT);
gpio_init(DIG2_GPIO); gpio_set_dir(
    DIG2_GPIO, GPIO_OUT);
dig_enable(false, false);
gpio_init(BUTTON_GPIO);
gpio_set_dir(BUTTON_GPIO, GPIO_IN);
gpio_pull_up(BUTTON_GPIO);

int value = 0;
bool up = true;

while (true) {
    if (button_clicked()) up = !up;
    int tens = value / 10;
    int ones = value % 10;
    for (int i = 0; i < 40; i++) {
        if (button_clicked()) up = !up;
        segments_write(tens);
        dig_enable(true, false);
        sleep_ms(5); dig_enable(
            false, false);
        segments_write(ones);
        dig_enable(false, true);
        sleep_ms(5); dig_enable(
            false, false);
    }
    if (up) value = (value == 99) ? 0
        : value + 1;
    else value = (value == 0) ? 99
        : value - 1;
    printf("Numero: %d | Sentido: %s\n"
        , value, up ? "SUBE" : "BAJA");
}
}
```

B. Archivo CMakeLists.txt

ANEXO B – CMakeLists.txt (completo)

```
cmake_minimum_required(VERSION 3.13)

include(pico_sdk_import.cmake)
set(PICO_BOARD pico_w)

project(PRACTICA_9)

pico_sdk_init()

add_executable(Displays
    Displays.c
)

target_link_libraries(Displays
```

```
pico_stdlib
)  
  
pico_add_extra_outputs(Displays)
```

C. Diagrama en Protoboard

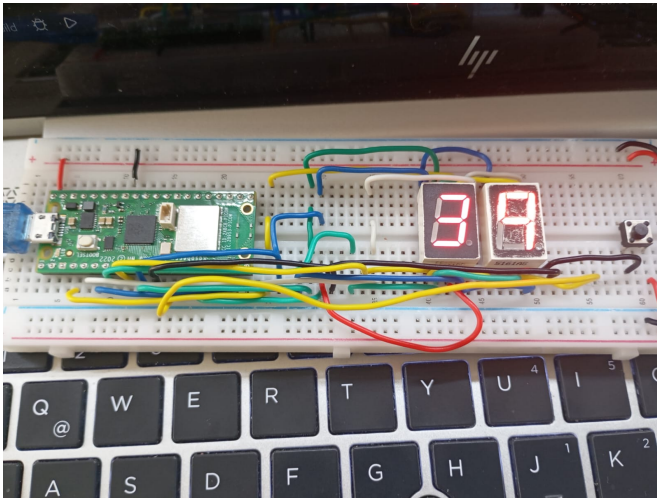


Figura 5: Diagrama en protoboard del contador bidireccional con dos displays multiplexados (Fritzing).